

ROYAUME DU MAROC
UNIVERSITE MOHAMMED PREMIER
ECOLE NATIONALE DES SCIENCES
APPLIQUEES
OUJDA



المملكة المغربية
جامعة محمد الأول
المدرسة الوطنية للعلوم
التطبيقية
وجدة

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES D'OUJDA

PROJET DE FIN D'ANNÉE (PFA)

Filière : Ingénierie Data Science et Cloud Computing

MCP-Powered BI Ecosystem

*Une plateforme décisionnelle généraliste multi-agents orchestrée par le
protocole MCP*

Réalisé par :

Mohamed Rouane
Zakariae Trachni

Encadré par :

Pr. Abdelmounaim Kerkri

Membres du jury :

Pr. Asmae El Mezouari
Pr. Abdelmounaim Kerkri

Année Universitaire 2025 – 2026

Dédicace

À nos chers **parents**,
pour leur amour inconditionnel, leurs sacrifices
et leur soutien qui n'a jamais faibli.

À nos **familles** et à nos **ami(e)s**,
pour leurs encouragements constants.

À nos **enseignants**,
qui ont éveillé en nous la curiosité
et le goût du travail bien fait.

À toutes celles et ceux qui croient
qu'une donnée, bien analysée, éclaire la décision.

Mohamed Rouane & Zakariae Trachni

Remerciements

« La connaissance s’acquiert par l’expérience, et se transmet par l’exemple. »

Au terme de ce projet de fin d’année, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont rendu ce travail possible.

Nos remerciements les plus sincères s’adressent à notre encadrant, **Pr. Abdelmounaim Kerkri**, qui a su transformer ce projet exigeant en une expérience formatrice remarquable grâce à :

- son **encadrement rigoureux** et ses orientations éclairées ;
- sa **disponibilité** constante et son écoute attentive ;
- ses **conseils avisés**, déterminants pour nos choix techniques ;
- son **exigence bienveillante**, qui nous a poussés à nous dépasser.

Son accompagnement a été déterminant pour :

- structurer notre démarche d’ingénierie ;
- approfondir notre compréhension des architectures multi-agents ;
- mener à bien la réalisation pratique de cette plateforme.

Nous adressons également notre reconnaissance :

- à l’ensemble du **corps professoral** de l’École Nationale des Sciences Appliquées d’Oujda, pour la qualité de la formation dispensée ;
- à nos **familles**, pour leur soutien indéfectible ;
- à nos **camarades de promotion**, pour leur entraide et leurs encouragements.

Ce projet a constitué une expérience académique enrichissante, permettant une application concrète des connaissances théoriques. Nous en garderons non seulement des compétences techniques solides, mais aussi l’exemple d’un enseignement de grande qualité.

Avec toute notre reconnaissance,

Mohamed Rouane & Zakariae Trachni

Résumé

Les outils de *Business Intelligence* (BI) traditionnels exigent un travail manuel important : définition des indicateurs, modélisation des données, conception des tableaux de bord et rédaction des rapports. Ce processus est long, coûteux et fortement dépendant d’experts. Ce projet propose **MCP-Powered BI Ecosystem**, une plateforme décisionnelle *généraliste* et *automatisée* capable d’analyser n’importe quel jeu de données tabulaire (CSV, Excel, JSON) *sans hypothèse préalable* sur les noms de colonnes ni sur le domaine métier.

La plateforme repose sur une équipe de **sept agents d’intelligence artificielle** spécialisés, orchestrés via le **protocole MCP** (*Model Context Protocol*). Un serveur MCP expose, sous forme d’outils contrôlés par permissions, l’ensemble des capacités du système ; chaque agent (Orchestrateur, Ingestion, Préparation, KPI, Tableau de bord, Publication, Rédaction) n’a accès qu’aux outils strictement nécessaires à son rôle. Un **moteur déterministe adaptatif** profile automatiquement les données, en déduit des indicateurs et des visualisations pertinents, tandis que les grands modèles de langage (LLM) curent, priorisent et rédigent l’analyse. Le système produit, par exécution, un **tableau de bord interactif**, un **rapport décisionnel** (page web et PDF) et un magasin d’artefacts reproductible.

Des mécanismes de **fiabilité** ont été conçus : filets de sécurité déterministes garantissant des livrables complets même en cas de défaillance d’un agent, vérification automatique des chiffres du rapport face aux indicateurs calculés, et suivi des coûts (jetons, latence). Une étude de cas sur un jeu de **17 769 matchs internationaux de football** valide l’approche : carte mondiale des buts, détection d’anomalies (dont le match historique Australie 31–0 Samoa américaines) et mise en évidence de l’avantage du terrain.

Mots-clés : Business Intelligence, agents IA, protocole MCP, grands modèles de langage, automatisation décisionnelle, visualisation de données, FastAPI, Plotly.

Abstract

Traditional Business Intelligence (BI) tools require substantial manual effort : defining indicators, modelling data, designing dashboards and writing reports. This process is slow, costly and heavily dependent on experts. This project introduces **MCP-Powered BI Ecosystem**, a *generalist* and *automated* decision-support platform able to analyse any tabular dataset (CSV, Excel, JSON) *without prior assumptions* about column names or business domain.

The platform relies on a team of **seven specialised AI agents** orchestrated through the **Model Context Protocol (MCP)**. An MCP server exposes every capability of the system as permission-controlled tools ; each agent (Orchestrator, Ingestion, Data Prep, KPI, Dashboard, Publishing, Reporter) can only access the tools strictly required by its role. A **deterministic adaptive engine** automatically profiles the data and derives relevant indicators and visualisations, while Large Language Models (LLMs) curate, prioritise and write the analysis. Each run yields an **interactive dashboard**, a **BI report** (web page and PDF) and a reproducible artifact store.

Reliability mechanisms were designed : deterministic safety nets guaranteeing complete deliverables even when an agent misbehaves, automatic verification of the report's figures against the computed indicators, and cost tracking (tokens, latency). A case study on **17,769 international football matches** validates the approach : a world map of goals, anomaly detection (including the historic Australia 31–0 American Samoa match) and the home-advantage effect.

Keywords : Business Intelligence, AI agents, MCP protocol, Large Language Models, decision automation, data visualisation, FastAPI, Plotly.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Introduction générale	1
1 Contexte général et état de l’art	3
1.1 Cadre du projet	3
1.2 La Business Intelligence et la chaîne décisionnelle	3
1.2.1 Définition et finalité	3
1.2.2 La chaîne décisionnelle	4
1.2.3 Notions d’indicateurs, dimensions et mesures	4
1.3 Problématique : les limites des outils BI classiques	4
1.4 Objectifs et périmètre	5
1.5 État de l’art	6
1.5.1 Les grands modèles de langage (LLM)	6
1.5.2 Les agents d’intelligence artificielle	6
1.5.3 Le protocole MCP (Model Context Protocol)	6
1.5.4 Modélisation dimensionnelle et entrepôts de données	7
1.5.5 Outils de visualisation et frameworks web	7
1.6 Méthodologie de travail	7
1.7 Gestion et planification du projet	7
2 Analyse et conception	9
2.1 Spécification des besoins	9
2.1.1 Besoins fonctionnels	9
2.1.2 Besoins non fonctionnels	9
2.1.3 Acteurs et cas d’usage	10
2.2 Architecture globale	10
2.3 Le serveur MCP et le registre d’outils	11
2.4 Les sept agents et la matrice de permissions	11
2.4.1 Rôles des agents	12

2.4.2	Boucle d'exécution d'un agent	13
2.5	Le moteur déterministe adaptatif	13
2.5.1	Profilage par rôles	13
2.5.2	Génération adaptative	13
2.6	Diagrammes de conception	13
2.6.1	Vue en composants	14
3	Réalisation et implémentation	16
3.1	Environnement et technologies	16
3.2	Interface utilisateur	16
3.3	Abstraction des fournisseurs de LLM	17
3.4	Ingestion, validation et profilage adaptatif	18
3.5	Calcul adaptatif des indicateurs (KPI)	19
3.6	Visualisations et tableau de bord	19
3.6.1	La carte géographique (choroplèthe)	20
3.6.2	Tendances temporelles	20
3.6.3	Comparaisons et distributions	20
3.7	Analyses avancées : anomalies et segments	21
3.7.1	Détection d'anomalies	21
3.7.2	Comparaison de segments	22
3.8	Orchestration et filets de sécurité	22
3.9	Publication des livrables	23
3.10	Fiabilité : véracité des chiffres et suivi des coûts	24
3.10.1	Vérification des chiffres du rapport	24
3.10.2	Suivi de la consommation	24
4	Tests, résultats et étude de cas	25
4.1	Stratégie de tests	25
4.2	Étude de cas : matchs internationaux de football	25
4.2.1	Présentation du jeu de données	25
4.2.2	Résultats : indicateurs et tableau de bord	26
4.2.3	Résultats : anomalies détectées	27
4.2.4	Résultats : avantage du terrain	27
4.2.5	Véracité du rapport	28
4.3	Discussion	28
4.3.1	Apports	28
4.3.2	Limites	28
	Conclusion générale et perspectives	30

Bibliographie	31
A Annexes	32
A.1 Guide d’installation et d’exécution	32
A.2 Extrait : matrice de permissions (<code>config.yaml</code>)	32
A.3 Extrait : boucle d’appel d’outils d’un agent	33
A.4 Arborescence du projet	33

Table des figures

1.1	La chaîne décisionnelle classique de la Business Intelligence.	4
1.2	Diagramme de Gantt simplifié du déroulement du projet.	8
2.1	Diagramme de cas d’usage de la plateforme.	10
2.2	Architecture en couches du système.	11
2.3	Diagramme de séquence simplifié d’une exécution du pipeline.	14
2.4	Diagramme de composants et dépendances.	14
3.1	Page d’accueil : téléversement, présentation des agents et historique.	17
3.2	Suivi en temps réel de l’exécution du pipeline et synthèse des agents.	18
3.3	Indicateurs clés calculés automatiquement (extrait du rapport).	19
3.4	Tableau de bord interactif : navigation, indicateurs et début de la carte.	20
3.5	Carte choroplèthe : buts à domicile cumulés par pays.	21
3.6	Buts à domicile au fil du temps.	21
3.7	Buts à l’extérieur au fil du temps.	21
3.8	Distribution des buts à domicile (histogramme).	22
3.9	Top tournois par buts à domicile.	22
3.10	Top équipes par buts à domicile.	22
3.11	Section « Anomalies » du rapport : valeurs aberrantes et exemple le plus extrême.	23
3.12	Section « Comparaison de segments » du rapport.	23
3.13	En-tête du rapport : navigation (Accueil, Dashboard) et export (PDF, HTML).	24
3.14	Conclusions du rapport avec le badge de vérification des chiffres.	24
4.1	Section « source des données » : schéma détecté automatiquement et validation.	27

Liste des tableaux

1.1	Comparaison entre la BI traditionnelle et l'approche proposée.	5
2.1	Les outils exposés par le serveur MCP.	12
2.2	Matrice rôles / permissions (extraite de <code>config.yaml</code>).	12
3.1	Technologies utilisées.	16
4.1	Couverture des tests automatisés.	26
4.2	Indicateurs clés calculés pour l'étude de cas.	28

Introduction générale

À l'ère du numérique, les organisations produisent et collectent des volumes de données sans précédent. Transformer ces données brutes en informations exploitables, puis en décisions éclairées, constitue l'enjeu central de la *Business Intelligence* (BI). Pourtant, malgré la maturité des outils du marché (tableurs avancés, plateformes comme Power BI ou Tableau), la chaîne décisionnelle demeure largement **manuelle** : un analyste doit comprendre le schéma des données, choisir les indicateurs clés de performance (KPI), concevoir des visualisations, puis rédiger un rapport de synthèse. Chaque nouveau jeu de données relance ce cycle coûteux en temps et en expertise.

L'émergence des **grands modèles de langage** (LLM) et des **agents d'intelligence artificielle** ouvre la voie à une automatisation beaucoup plus poussée de cette chaîne. Toutefois, confier l'intégralité de l'analyse à un modèle de langage se heurte à deux écueils : les modèles *hallucinent* (ils inventent des chiffres ou des colonnes inexistantes) et peinent à manipuler de manière fiable des jeux de données volumineux. À l'inverse, une automatisation purement algorithmique produit des analyses rigides et dépourvues de discernement métier.

C'est dans cette tension que s'inscrit notre projet de fin d'année. Nous proposons **MCP-Powered BI Ecosystem**, un écosystème décisionnel qui réconcilie ces deux mondes : un **moteur déterministe** garantit l'exactitude des calculs et la robustesse face à n'importe quel schéma de données, tandis qu'une **équipe d'agents IA** apporte le jugement, la curation et la narration. L'ensemble est orchestré par le **protocole MCP** (*Model Context Protocol*), un standard récent qui expose les capacités du système comme un ensemble d'outils contrôlés par permissions et mobilisables par les agents.

L'originalité de l'approche tient à son caractère **généraliste** : le système ne fait *aucune* hypothèse sur le domaine métier. Qu'il s'agisse de ventes, de logistique, de ressources humaines ou de données sportives, la plateforme découvre seule la structure des données, en déduit les indicateurs et visualisations pertinents, puis produit un tableau de bord interactif et un rapport décisionnel complet.

Problématique

Comment concevoir une plateforme de Business Intelligence capable d'analyser **automatiquement** et de manière **fiable** n'importe quel jeu de données tabulaire, en combinant la rigueur d'un moteur déterministe et le discernement d'agents fondés sur des modèles de langage, tout en garantissant la traçabilité, la maîtrise des coûts et la véracité des

résultats produits ?

Objectifs

Le projet poursuit les objectifs suivants :

- concevoir une architecture **multi-agents** orchestrée par un serveur MCP, avec un contrôle d'accès par rôle ;
- développer un **moteur adaptatif** de profilage, de calcul d'indicateurs et de génération de visualisations, indépendant du domaine métier ;
- produire automatiquement un **tableau de bord interactif** et un **rapport décisionnel** (web et PDF) reproductibles ;
- intégrer des mécanismes de **fiabilité** : filets de sécurité, vérification des chiffres, suivi des coûts ;
- valider l'approche par une **étude de cas** réelle.

Organisation du rapport

Ce rapport s'articule autour de quatre chapitres. Le **chapitre 1** présente le contexte général, la problématique et l'état de l'art (BI, LLM, agents, protocole MCP). Le **chapitre 2** détaille l'analyse des besoins et la conception de l'architecture. Le **chapitre 3** décrit la réalisation et l'implémentation des différents modules. Enfin, le **chapitre 4** expose la stratégie de tests, les résultats obtenus et une étude de cas approfondie, avant la conclusion et les perspectives.

Chapitre 1

Contexte général et état de l’art

En bref

Ce chapitre situe le projet dans son cadre académique, rappelle les fondements de la Business Intelligence et de sa chaîne décisionnelle, expose les limites des outils existants, puis dresse l’état de l’art des briques technologiques mobilisées : grands modèles de langage, agents d’IA et protocole MCP.

1.1 Cadre du projet

Ce travail s’inscrit dans le cadre du **Projet de Fin d’Année (PFA)** de la filière *Ingénierie Data Science et Cloud Computing* à l’**École Nationale des Sciences Appliquées d’Oujda**. Le PFA constitue un jalon pédagogique majeur : il vise à confronter les étudiants à un problème d’ingénierie réaliste, à mobiliser de façon intégrée les connaissances acquises (programmation, science des données, génie logiciel, intelligence artificielle) et à développer des compétences de conception, de gestion de projet et de communication scientifique.

Le sujet retenu se situe à l’intersection de trois domaines en pleine expansion : la **Business Intelligence**, l’**ingénierie des données** et l’**IA générative**. Il répond à une préoccupation concrète des organisations : réduire le temps et l’expertise nécessaires pour transformer des données brutes en connaissances décisionnelles.

1.2 La Business Intelligence et la chaîne décisionnelle

1.2.1 Définition et finalité

La **Business Intelligence** (informatique décisionnelle) désigne l’ensemble des méthodes, des processus et des technologies permettant de collecter, d’organiser, d’analyser et de restituer les données d’une organisation afin d’éclairer la prise de décision. Son objectif n’est pas la donnée pour elle-même, mais la *connaissance actionnable* : répondre aux questions « que s’est-il passé ? », « pourquoi ? » et « que faire ? ».

1.2.2 La chaîne décisionnelle

La création de valeur à partir des données suit classiquement une chaîne en plusieurs maillons, illustrée par la figure 1.1.

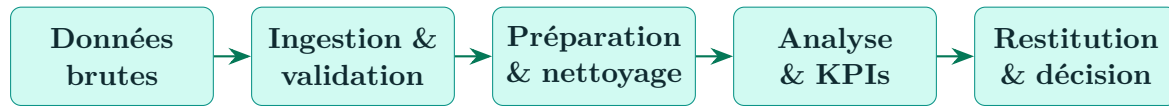


Figure 1.1 – La chaîne décisionnelle classique de la Business Intelligence.

Chaque maillon mobilise des savoir-faire distincts :

- **Ingestion** : récupération des données depuis des sources hétérogènes (fichiers, bases de données, API) et contrôle de leur intégrité ;
- **Préparation** : nettoyage, traitement des valeurs manquantes, typage, normalisation ;
- **Analyse** : définition et calcul des indicateurs clés de performance (KPI), exploration statistique ;
- **Restitution** : conception de tableaux de bord et de rapports destinés aux décideurs.

1.2.3 Notions d'indicateurs, dimensions et mesures

La modélisation décisionnelle distingue traditionnellement les **mesures** (grandeurs numériques agrégeables : chiffre d'affaires, quantités, buts marqués) des **dimensions** (axes d'analyse catégoriels ou temporels : région, produit, date) selon lesquels les mesures sont ventilées. Un **KPI** (*Key Performance Indicator*) est une mesure synthétique, souvent agrégée selon une ou plusieurs dimensions, choisie pour sa pertinence métier. Cette distinction, fondamentale, est au cœur de l'approche adaptative développée dans ce projet (chapitre 3).

1.3 Problématique : les limites des outils BI classiques

Malgré leur richesse fonctionnelle, les solutions de BI du marché présentent plusieurs limites structurantes :

Forte dépendance à l'expertise. La conception d'un tableau de bord pertinent suppose une bonne compréhension métier *et* technique. L'analyste reste un goulot d'étranglement.

Travail répétitif et chronophage. Chaque nouveau jeu de données impose de reparcourir la chaîne décisionnelle : exploration, choix des indicateurs, construction des graphiques, rédaction.

Spécialisation par cas d’usage. Les solutions sont souvent paramétrées pour un domaine précis (BI commerciale, BI RH), avec des hypothèses figées sur la nature des colonnes.

Restitution manuelle. La rédaction du rapport narratif — pourtant essentielle à la décision — demeure entièrement humaine.

Ces limites motivent la recherche d’une plateforme **généraliste** et **automatisée**, capable de s’adapter à n’importe quel schéma de données et de produire, sans intervention humaine, l’ensemble des livrables décisionnels. Le tableau 1.1 synthétise l’écart entre l’approche traditionnelle et celle que nous proposons.

Table 1.1 – Comparaison entre la BI traditionnelle et l’approche proposée.

Critère	BI traditionnelle	Approche proposée
Choix des KPIs	Manuel, par un expert	Automatique et adaptatif
Conception des graphiques	Manuelle	Automatique selon les rôles
Rédaction du rapport	Humaine	Générée par agents IA
Adaptation au domaine	Paramétrage spécifique	Aucune hypothèse métier
Délai d’obtention	Heures à jours	Quelques secondes
Véracité des chiffres	Confiance humaine	Vérification automatique

1.4 Objectifs et périmètre

Le périmètre du projet couvre l’intégralité de la chaîne décisionnelle, de l’ingestion d’un fichier tabulaire jusqu’à la production d’un rapport décisionnel. Les choix structurants sont les suivants :

- support des formats CSV, Excel et JSON ;
- **aucune** configuration spécifique au domaine métier ;
- génération automatique d’un tableau de bord interactif et d’un rapport (web + PDF) ;
- exécution locale, reproductible, avec un magasin d’artefacts par exécution.

Sont hors périmètre : la connexion temps réel à des bases de données de production, la gestion multi-utilisateurs et le déploiement à l’échelle, qui constituent des perspectives (voir la conclusion générale).

1.5 État de l'art

1.5.1 Les grands modèles de langage (LLM)

Un **grand modèle de langage** est un réseau de neurones, fondé sur l'architecture *Transformer* [7], entraîné sur d'immenses corpus textuels pour prédire le mot suivant. Au-delà de la génération de texte, ces modèles manifestent des capacités de raisonnement, de synthèse et de *suivi d'instructions*. Une avancée déterminante pour notre projet est l'**appel d'outils** (*tool use / function calling*) : le modèle peut décider d'invoquer une fonction externe avec des arguments structurés, puis exploiter le résultat. C'est précisément ce mécanisme qui permet à un LLM de piloter un système logiciel plutôt que de se contenter de produire du texte.

Nous mobilisons des modèles performants tels que **Llama 3.3** [4] servis par l'infrastructure **Groq** (inférence à très faible latence), ou des modèles exécutés localement via **Ollama**. Les LLM présentent néanmoins deux faiblesses notoires : la tendance à l'**hallucination** (production d'informations erronées mais plausibles) et la difficulté à manipuler fidèlement de grandes tables. Ces limites guident notre choix architectural : confier les calculs à un moteur déterministe et réserver aux LLM la curation et la rédaction.

1.5.2 Les agents d'intelligence artificielle

Un **agent** est un système qui perçoit un environnement, raisonne et agit pour atteindre un objectif. Appliqué aux LLM, le paradigme agentique consiste à doter un modèle d'une **boucle** : observer l'état, décider d'une action (souvent un appel d'outil), exécuter, observer le résultat, et recommencer jusqu'à l'accomplissement de la tâche. Les approches multi-agents décomposent un problème complexe en **rôles spécialisés** qui coopèrent, à l'image d'une équipe humaine. Cette spécialisation améliore la fiabilité, la lisibilité et la maintenabilité, chaque agent disposant d'un contexte et de permissions restreints à sa mission.

1.5.3 Le protocole MCP (Model Context Protocol)

Le **Model Context Protocol** [1] est un standard ouvert, introduit par Anthropic, qui normalise la manière dont une application expose ses **outils**, ses **ressources** et ses **données** à des modèles de langage. À la manière d'un « port universel » entre les LLM et le monde logiciel, MCP définit un format commun pour décrire un outil (nom, description, schéma JSON des arguments) et pour l'invoquer.

Dans notre projet, MCP joue un double rôle structurant : il sert de **registre central des capacités** du système (chargement de données, profilage, calcul d'indicateurs,

génération de graphiques, etc.) et de **mécanisme de contrôle d’accès**. Chaque agent se voit attribuer un sous-ensemble d’outils autorisés ; toute tentative d’appel hors de ce périmètre est refusée par le serveur. Ce principe de *moindre privilège* renforce la sûreté et clarifie les responsabilités.

1.5.4 Modélisation dimensionnelle et entrepôts de données

La théorie de la **modélisation dimensionnelle**, formalisée notamment par Kimball [2], structure les données décisionnelles autour de *tables de faits* (contenant les mesures) et de *tables de dimensions* (contenant les axes d’analyse). Cette séparation conceptuelle entre **faits/mesures** et **dimensions** irrigue directement notre moteur de profilage : plutôt que d’imposer un schéma en étoile prédéfini, le système *infère* dynamiquement le rôle de chaque colonne. Cette inférence automatique des rôles constitue la transposition « sans configuration » des principes de la modélisation dimensionnelle à un contexte généraliste.

1.5.5 Outils de visualisation et frameworks web

La restitution s’appuie sur **Plotly** [5], une bibliothèque de visualisation interactive produisant des graphiques en HTML/JavaScript autonomes, et sur **FastAPI** [6], un framework web Python moderne et performant pour l’interface utilisateur. La manipulation des données repose sur **pandas** [3], référence de l’analyse de données tabulaires en Python.

1.6 Méthodologie de travail

Le projet a été conduit selon une démarche **itérative et incrémentale**. Une première itération a établi le socle (moteur déterministe, serveur MCP, sept agents, interface) ; les itérations suivantes ont enrichi le système (robustesse, vérification des chiffres, suivi des coûts, cartes géographiques, détection d’anomalies, comparaison de segments) et amélioré l’interface. Chaque incrément a été **validé par des tests** automatisés et par des exécutions de bout en bout sur des jeux de données réels, garantissant la non-régression. Cette approche a permis de livrer en permanence un système fonctionnel et de prioriser les fonctionnalités à plus forte valeur.

1.7 Gestion et planification du projet

Le projet a été planifié en **cinq phases** successives mais partiellement recouvrantes, conformément à la démarche itérative. La figure 1.2 en présente le diagramme de Gantt simplifié.

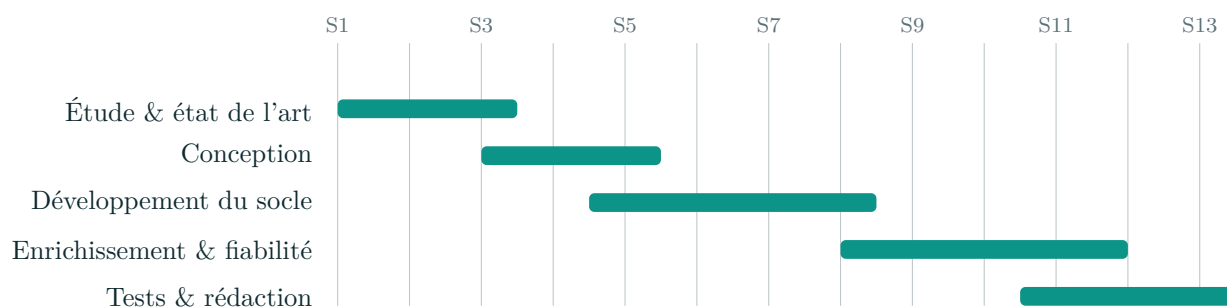


Figure 1.2 – Diagramme de Gantt simplifié du déroulement du projet.

Étude & état de l’art. Cadrage du sujet, revue des concepts (BI, LLM, agents, MCP) et choix technologiques.

Conception. Définition des besoins, de l’architecture en couches et de la répartition des rôles entre agents.

Développement du socle. Moteur déterministe, serveur MCP, sept agents, interface web et premier pipeline de bout en bout.

Enrichissement & fiabilité. Filets de sécurité, vérification des chiffres, suivi des coûts, cartes géographiques et analyses avancées.

Tests & rédaction. Consolidation des tests, étude de cas et rédaction du présent rapport.

Synthèse du chapitre

La BI traditionnelle reste manuelle, coûteuse et spécialisée. Les LLM et les agents, couplés au protocole MCP, offrent les briques d’une automatisation généralisée — à condition d’en maîtriser les faiblesses (hallucinations, fiabilité). Le chapitre suivant traduit ce constat en une architecture concrète.

Chapitre 2

Analyse et conception

En bref

Ce chapitre formalise les besoins, présente l’architecture en couches du système, détaille le serveur MCP et son registre d’outils permissionné, décrit les sept agents et leur matrice de permissions, expose le moteur déterministe adaptatif, et illustre la conception par des diagrammes UML.

2.1 Spécification des besoins

2.1.1 Besoins fonctionnels

Le système doit permettre à un utilisateur de :

1. **téléverser** un jeu de données tabulaire (CSV, Excel, JSON) ;
2. déclencher une **analyse automatique** de bout en bout ;
3. suivre en **temps réel** l’avancement du traitement par étape ;
4. consulter un **tableau de bord interactif** (indicateurs et graphiques) ;
5. consulter et **exporter** un **rapport décisionnel** (page web, PDF, HTML) ;
6. retrouver et rouvrir les **exécutions passées** et leurs artefacts.

2.1.2 Besoins non fonctionnels

Généralité. Le système ne doit faire aucune hypothèse sur le domaine métier ni sur les noms de colonnes.

Fiabilité. Une exécution doit toujours produire des livrables complets, même en cas de défaillance d’un agent.

Véracité. Les chiffres du rapport doivent être vérifiables face aux indicateurs effectivement calculés.

Sûreté. Chaque agent ne doit accéder qu’aux outils nécessaires à son rôle (moindre privilège).

Reproductibilité. Chaque exécution doit être tracée dans un magasin d’artefacts autonome.

Observabilité. La consommation (jetons, latence, coût estimé) doit être mesurée et restituée.

Portabilité. Le système doit fonctionner avec un fournisseur de LLM hébergé (Groq) ou local (Ollama).

2.1.3 Acteurs et cas d’usage

L’acteur principal est l’**Analyste / Décideur**, qui interagit avec la plateforme via une interface web. La figure 2.1 présente le diagramme de cas d’usage.

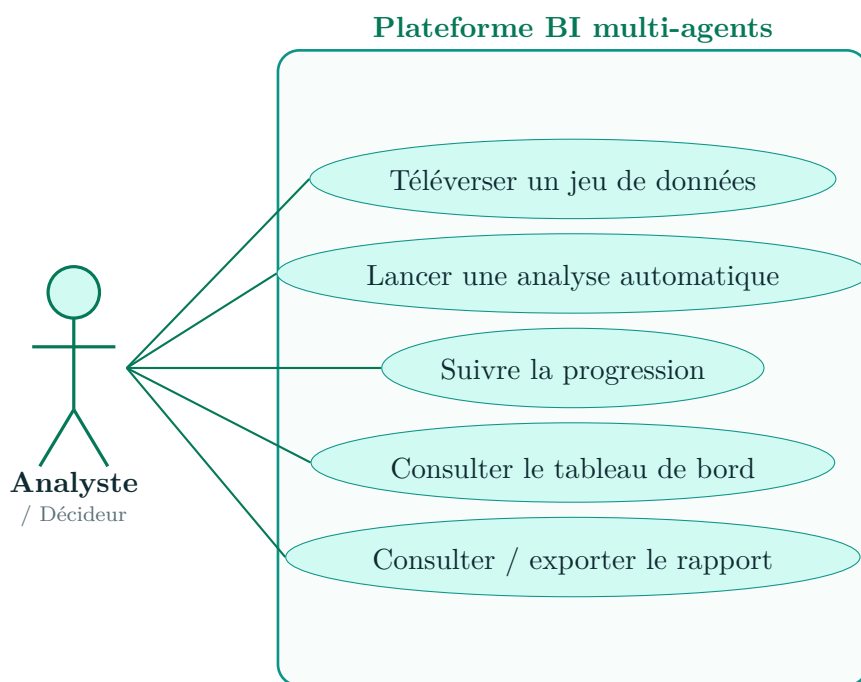


Figure 2.1 – Diagramme de cas d’usage de la plateforme.

2.2 Architecture globale

L’architecture adopte une organisation **en couches** (figure 2.2), qui sépare nettement les responsabilités et permet une évolution indépendante de chaque niveau.

Le principe directeur est une **séparation des préoccupations** :

- la **présentation** (interface web) ne contient aucune logique d’analyse ;
- l’**orchestration** séquence les agents et garantit la complétude des résultats ;
- les **agents** portent le jugement métier (curation, narration) ;
- le **serveur MCP** médie tous les accès aux capacités et applique les permissions ;
- le **moteur déterministe** effectue tous les calculs ;
- le **magasin d’artefacts** assure la traçabilité.

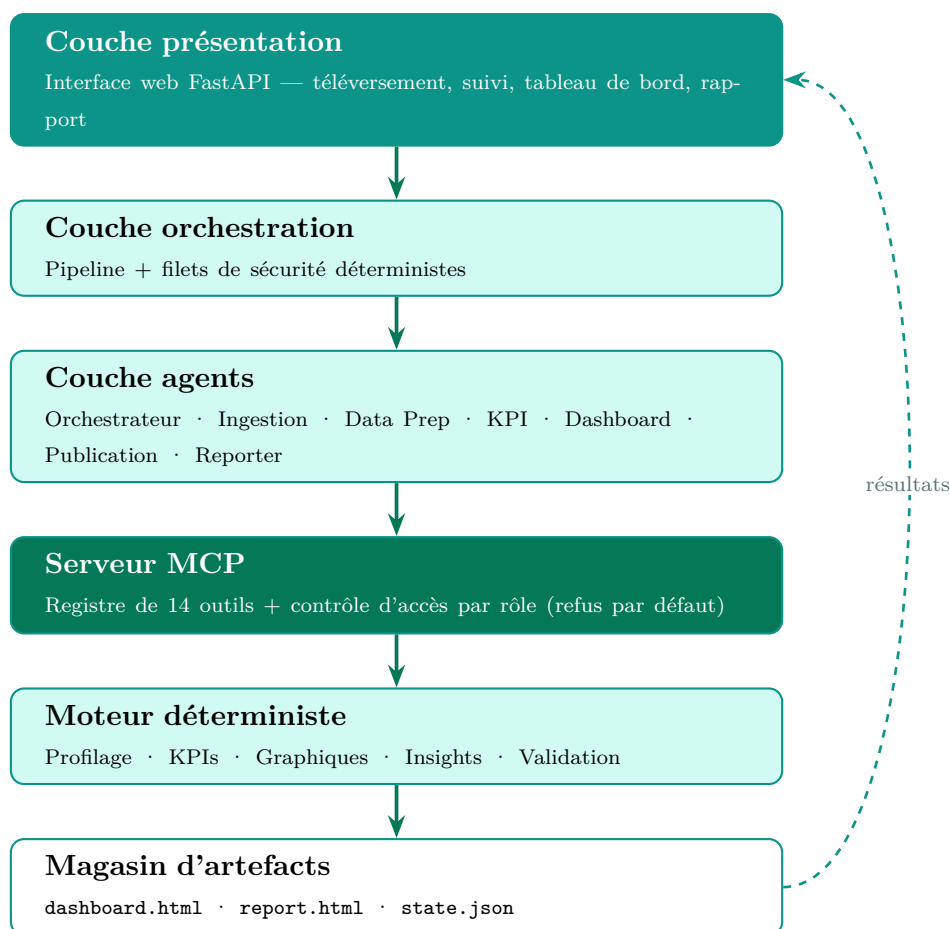


Figure 2.2 – Architecture en couches du système.

2.3 Le serveur MCP et le registre d'outils

Le serveur MCP est le **point de passage obligé** entre les agents et les capacités du système. Il expose 14 **outils**, chacun décrit par un nom, une description en langage naturel et un **schéma JSON** de ses arguments — description directement exploitable par les LLM pour décider de leurs appels. Le tableau 2.1 récapitule ces outils.

2.4 Les sept agents et la matrice de permissions

Le système répartit le travail entre **sept agents spécialisés**. Chaque agent possède un **rôle**, une **invite système** (*prompt*) qui le cadre, et un **ensemble d'outils autorisés** défini dans le fichier de configuration `config.yaml`. Le serveur MCP applique ces permissions à chaque appel, selon un principe de **refus par défaut**. Le tableau 2.2 présente cette matrice.

Table 2.1 – Les outils exposés par le serveur MCP.

Outil	Rôle
load_dataset	Charge un fichier (CSV/Excel/JSON) et l'enregistre.
validate_dataset	Contrôle structurel (doublons, colonnes vides...).
profile_dataset	Profile les colonnes (rôles, cardinalité, statistiques).
clean_dataset	Applique des opérations de nettoyage.
suggest_kpis / compute_kpis	Propose puis calcule les indicateurs.
detect_anomalies	Détecte les valeurs aberrantes et pics temporels.
compare_segments	Compare une mesure entre groupes.
suggest_charts / build_chart	Propose puis construit les graphiques.
build_dashboard	Assemble le tableau de bord interactif.
publish_artifacts	Fige l'état de l'exécution sur disque.
generate_report	Produit le rapport décisionnel (HTML).
get_run_state / set_run_state	Lit / écrit l'état partagé de l'exécution.

Table 2.2 – Matrice rôles / permissions (extraite de `config.yaml`).

Agent	Outils autorisés
Orchestrateur	list_agents, get_run_state, set_run_state
Ingestion	load_dataset, validate_dataset
Data Prep	profile_dataset, clean_dataset
KPI	profile_dataset, suggest_kpis, compute_kpis, detect_anomalies, compare_segments
Dashboard	profile_dataset, suggest_charts, build_chart
Publication	build_dashboard, publish_artifacts
Reporter	get_run_state, generate_report

2.4.1 Rôles des agents

Orchestrateur. Planifie l'exécution, observe la forme des données et signale les risques.

Il ne manipule pas directement les données.

Ingestion. Charge le fichier brut et valide sa structure.

Data Prep. Profile les colonnes et applique un nettoyage minimal et sûr.

KPI. Sélectionne, parmi des candidats adaptatifs, les indicateurs les plus pertinents, les calcule, puis détecte anomalies et écarts de segments.

Dashboard. Choisit un ensemble équilibré de graphiques et les construit.

Publication. Assemble le tableau de bord HTML et fige les artefacts.

Reporter. Rédige le rapport décisionnel en s'appuyant exclusivement sur l'état réel de l'exécution.

2.4.2 Boucle d'exécution d'un agent

Chaque agent fonctionne selon une **boucle d'appel d'outils** bornée : le LLM reçoit l'invite et la liste de ses outils autorisés, décide d'un ou plusieurs appels, le serveur MCP exécute et renvoie les résultats, puis le modèle poursuit jusqu'à produire une réponse finale. Un mécanisme de **relance avec incitation** (*retry-with-nudge*) corrige le cas fréquent où le modèle répond par du texte sans appeler ses outils.

2.5 Le moteur déterministe adaptatif

Le cœur « intelligent » généraliste ne peut reposer entièrement sur le LLM : face à des dizaines de colonnes, le modèle hallucinerait des noms et manquerait des agrégations évidentes. Le **moteur déterministe** produit donc une base adaptative riche, que les agents *curent* et *narrent*.

2.5.1 Profilage par rôles

Le profileur classe chaque colonne dans un **rôle sémantique** : **mesure** (numérique agrégeable), **dimension** (catégorielle de faible cardinalité), **date**, **identifiant**, **texte**, **booléen**, ou **géographie** (valeurs reconnues comme des pays). Ces rôles, déduits par des heuristiques paramétrables, permettent à tout le reste du système d'agir de manière générique.

2.5.2 Génération adaptative

À partir des seuls rôles, le moteur dérive :

- des **KPIs** candidats (totaux, moyennes, comptages distincts, variation période sur période, part du segment dominant) ;
- des **graphiques** candidats (séries temporelles, barres top-N, distributions, nuages de points, parts, carte choroplèthe, heatmap de corrélations) ;
- des **insights** (anomalies, comparaisons de segments, corrélations).

2.6 Diagrammes de conception

La figure 2.3 présente le diagramme de séquence simplifié d'une exécution, mettant en évidence l'orchestration des sept agents via le serveur MCP.

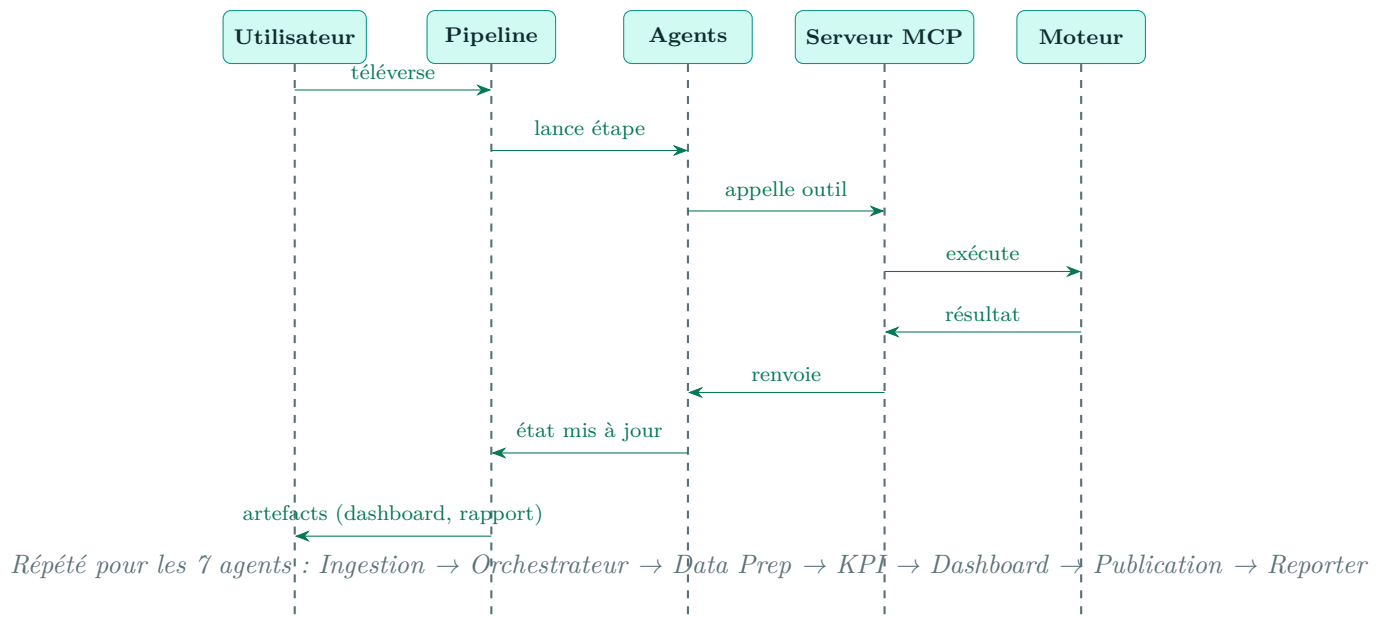


Figure 2.3 – Diagramme de séquence simplifié d'une exécution du pipeline.

2.6.1 Vue en composants

La figure 2.4 présente l'organisation en composants logiciels et leurs dépendances. Le pipeline dépend des agents, qui dépendent tous du serveur MCP ; celui-ci est le seul à dépendre du moteur déterministe et du client LLM. Cette dépendance *en entonnoir* matérialise le rôle de médiateur unique du serveur MCP.

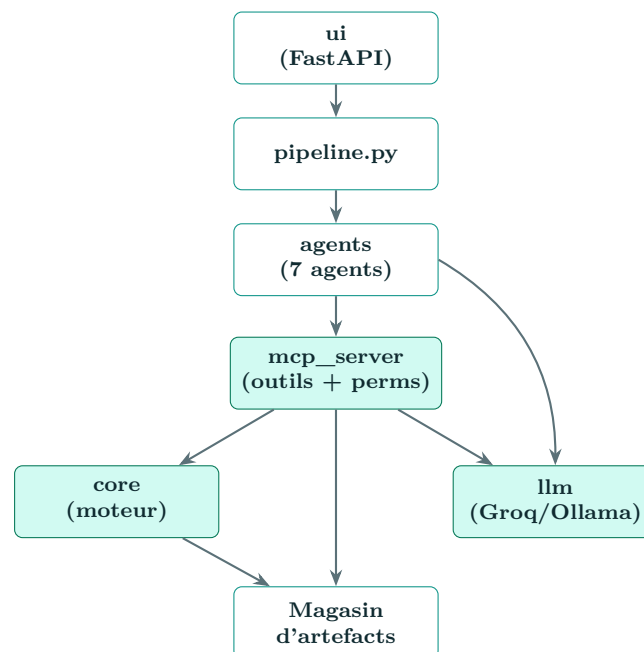


Figure 2.4 – Diagramme de composants et dépendances.

Synthèse du chapitre

L'architecture en couches isole présentation, orchestration, agents, serveur MCP, moteur et artefacts. Le serveur MCP centralise 14 outils et applique une matrice de permissions stricte aux sept agents. Le moteur déterministe garantit la généralité ; les agents apportent le jugement. Le chapitre suivant détaille la réalisation.

Chapitre 3

Réalisation et implémentation

En bref

Ce chapitre décrit la mise en œuvre concrète du système : l’environnement technique, le profilage adaptatif, le calcul des indicateurs, les visualisations (dont la carte géographique), les analyses avancées (anomalies, segments), l’orchestration avec ses filets de sécurité, la publication des livrables et les mécanismes de fiabilité.

3.1 Environnement et technologies

Le système est développé en **Python 3**. Le tableau 3.1 récapitule les principales technologies et leur rôle.

Table 3.1 – Technologies utilisées.

Technologie	Rôle dans le projet
pandas / NumPy	Manipulation et analyse des données tabulaires.
Plotly	Génération des graphiques interactifs (HTML autonome).
FastAPI / Uvicorn	Interface web (téléversement, suivi, restitution).
Jinja2	Gabarits HTML de l’interface.
mcp	Implémentation du serveur Model Context Protocol.
Groq / Ollama	Fournisseurs de LLM (hébergé / local).
kaleido	Export statique des graphiques (figures du présent rapport).

L’organisation du code suit la séparation en couches : le paquet `src/core/` contient le moteur déterministe, `src/llm/` l’abstraction des fournisseurs de LLM, `src/mcp_server/` le serveur MCP et ses outils, `src/agents/` les sept agents, et `src/ui/` l’interface web. Le pipeline (`src/pipeline.py`) relie l’ensemble.

3.2 Interface utilisateur

L’interface web, développée avec FastAPI et Jinja2, constitue le point d’entrée. La figure 3.1 présente la page d’accueil : zone de téléversement, présentation des sept agents et historique des exécutions passées.

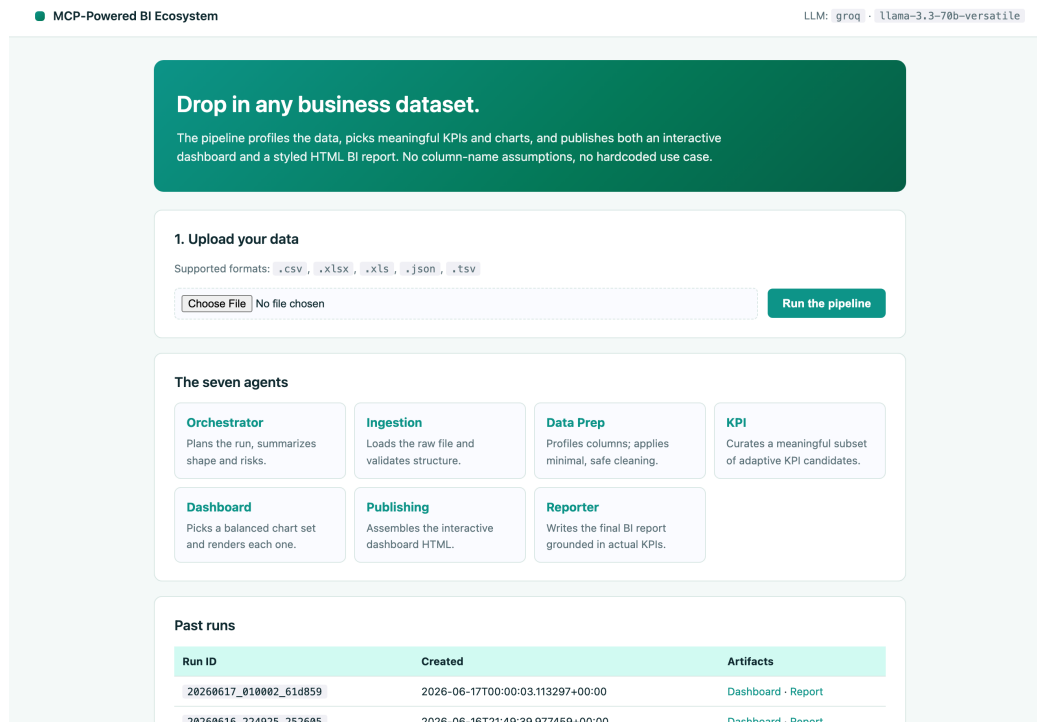


Figure 3.1 – Page d'accueil : téléversement, présentation des agents et historique.

Après téléversement, le traitement s'exécute en arrière-plan et l'utilisateur suit sa **progression par étape**. La figure 3.2 montre la vue d'exécution : statut de chaque agent, indicateurs de consommation (jetons, coût estimé, latence) et résultat de la vérification des chiffres.

3.3 Abstraction des fournisseurs de LLM

Pour garantir la portabilité, l'accès aux modèles de langage passe par une **abstraction** unique (`src/llm/client.py`) qui masque les différences entre fournisseurs. Une fabrique sélectionne, selon la configuration, le client **Groq** (hébergé, faible latence) ou **Ollama** (local, sans clé d'API). Les deux exposent la même méthode `chat()` acceptant des outils au format OpenAI et renvoyant un message normalisé, enrichi de la consommation (jetons) et de la latence. Ce dernier point alimente le suivi des coûts (section 3.10.2).

```

1 def get_llm_client():
2     provider = os.getenv("LLM_PROVIDER", "groq").lower()
3     if provider == "groq":
4         return GroqClient(api_key=os.getenv("GROQ_API_KEY"),
5                             model=os.getenv("GROQ_MODEL"))
6     if provider == "ollama":
7         return OllamaClient(host=os.getenv("OLLAMA_HOST"),
8                               model=os.getenv("OLLAMA_MODEL"))
9     raise RuntimeError(f"Fournisseur inconnu : {provider!r}")

```

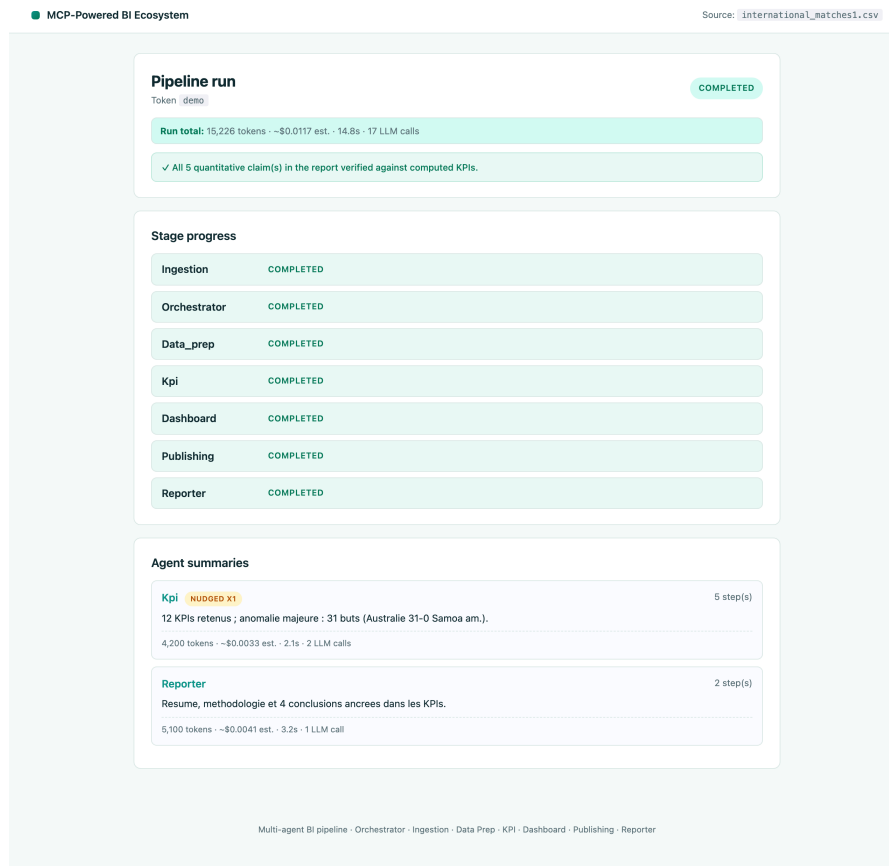


Figure 3.2 – Suivi en temps réel de l’exécution du pipeline et synthèse des agents.

Extrait de code 3.1 – Sélection du fournisseur de LLM (llm/client.py).

Cette conception applique le principe d’**inversion de dépendance** : les agents dépendent d’une interface abstraite (LLMClient) et non d’un fournisseur concret, ce qui permet d’en changer sans modifier le reste du code.

3.4 Ingestion, validation et profilage adaptatif

L’agent d’**Ingestion** charge le fichier (l’extension détermine le lecteur pandas approprié) puis valide sa structure (doublons, colonnes entièrement vides, nombre de lignes). Le profilage, cœur de la généralité du système, attribue à chaque colonne un rôle sémantique. L’extrait 3.2 illustre l’inférence du rôle pour les colonnes textuelles, où la détection **géographique** est testée en priorité.

```

1 # 4. Chaîne de caracteres / objet
2 else:
3     # La geographie est testee en premier et prime sur la cardinalite :
4     # une colonne de noms de pays est geographique meme tres cardinale.
5     if _looks_like_geo(s, geo_match_threshold):

```

```

6         role = ColumnRole.GEO
7     elif _looks_like_identifiier(str(col), card, row_count,
8         is_numeric=False):
9         role = ColumnRole.IDENTIFIER
10    elif card <= categorical_max_cardinality:
11        role = ColumnRole.DIMENSION
12    elif card > categorical_max_cardinality and card / max(row_count, 1)
13        > 0.5:
14        role = ColumnRole.TEXT
15    else:
16        role = ColumnRole.HIGH_CARD_DIM

```

Extrait de code 3.2 – Inférence de rôle pour une colonne textuelle (`profiler.py`).

La détection géographique compare les valeurs distinctes de la colonne à un référentiel d'environ 250 noms de pays ; si une fraction suffisante correspond (seuil paramétrable, 0,6 par défaut), la colonne devient une dimension géographique exploitable par la carte.

3.5 Calcul adaptatif des indicateurs (KPI)

L'agent **KPI** récupère une liste généreuse de KPIs candidats déduits du profil, en **cure** un sous-ensemble pertinent (jugement métier du LLM), puis les calcule de façon déterministe. Chaque KPI est décrit par une spécification JSON (identifiant, titre, type, mesure, agrégation, format), ce qui permet un calcul générique. La figure 3.3 montre les cartes d'indicateurs telles qu'elles apparaissent dans le rapport.

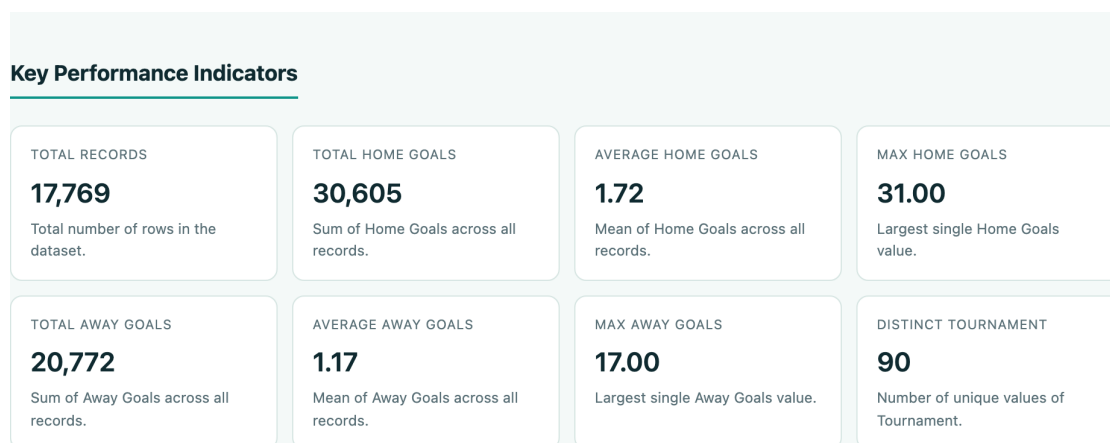


Figure 3.3 – Indicateurs clés calculés automatiquement (extrait du rapport).

3.6 Visualisations et tableau de bord

L'agent **Dashboard** sélectionne un ensemble **équilibré** de graphiques (tendance, comparaison, distribution, composition, carte) et les construit avec Plotly. Le moteur choisit

automatiquement le type de graphique selon les rôles des colonnes. La figure 3.4 présente le haut du tableau de bord généré.

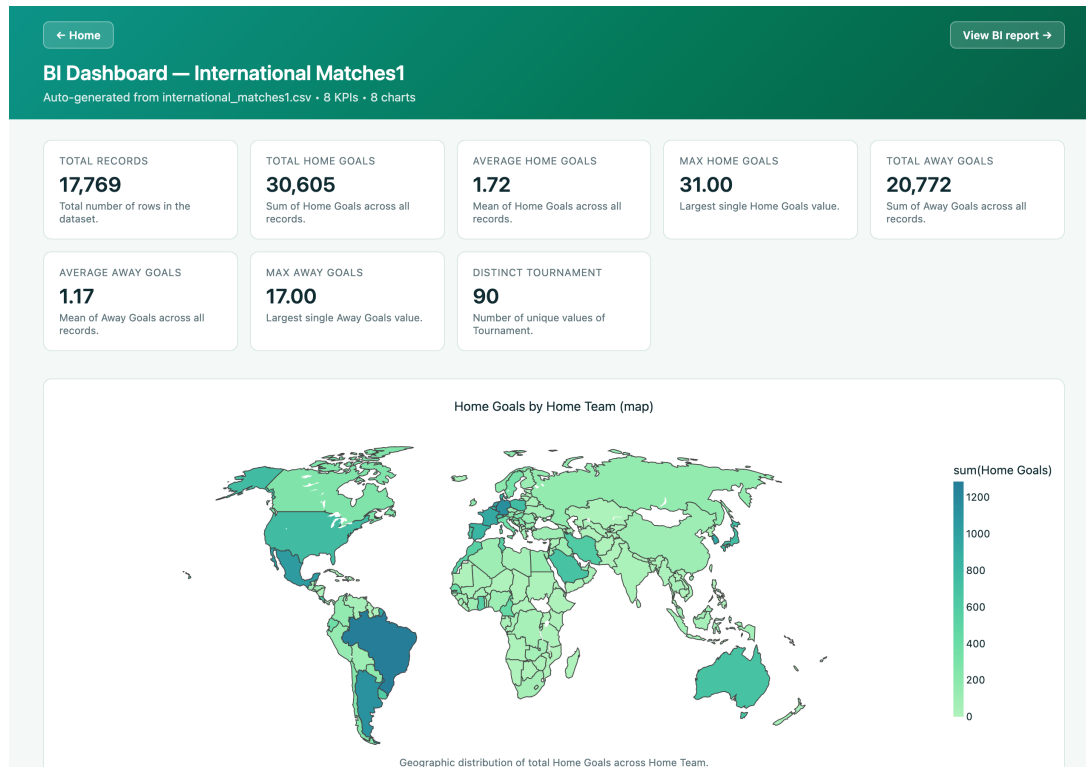


Figure 3.4 – Tableau de bord interactif : navigation, indicateurs et début de la carte.

3.6.1 La carte géographique (choroplèthe)

Lorsqu’une colonne géographique est détectée, le moteur produit une **carte choroplèthe** mondiale agrégeant une mesure par pays. La figure 3.5 en donne un exemple : la distribution des buts à domicile cumulés par pays.

3.6.2 Tendances temporelles

Lorsqu’une colonne de type date est présente, le moteur génère des séries temporelles, en agrégeant automatiquement par période adaptée au volume de points. Les figures 3.6 et 3.7 illustrent l’évolution des buts à domicile et à l’extérieur.

3.6.3 Comparaisons et distributions

Les graphiques en barres top-N classent les valeurs d’une dimension par mesure (figures 3.9 et 3.10), tandis que les **histogrammes** révèlent la distribution des mesures (figure 3.8), faisant apparaître l’asymétrie typique du nombre de buts (forte concentration sur les faibles valeurs, longue traîne).

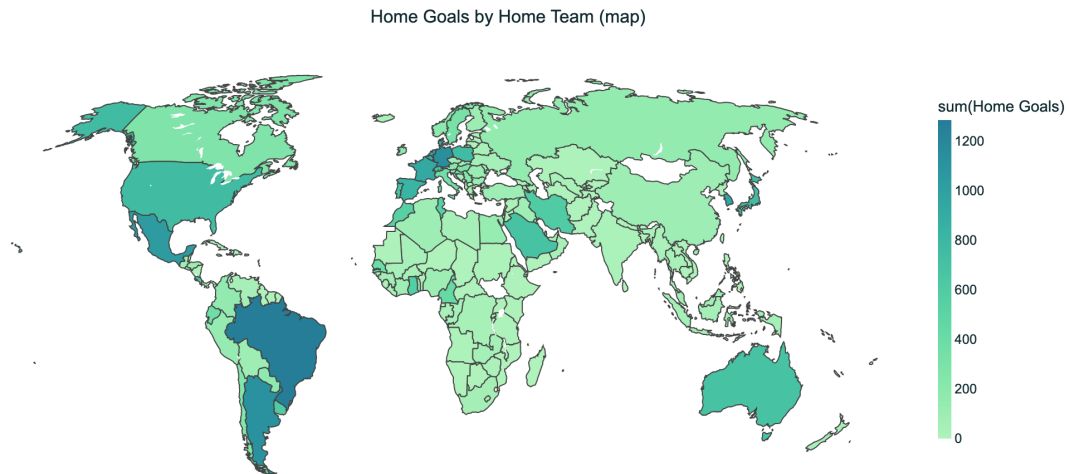


Figure 3.5 – Carte choroplèthe : buts à domicile cumulés par pays.

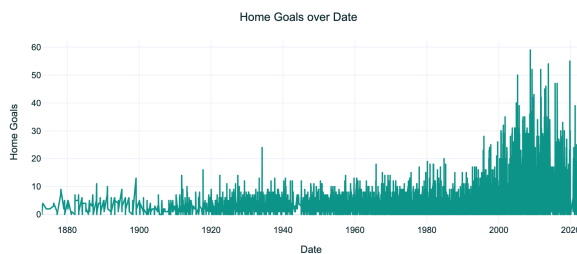


Figure 3.6 – Buts à domicile au fil du temps.

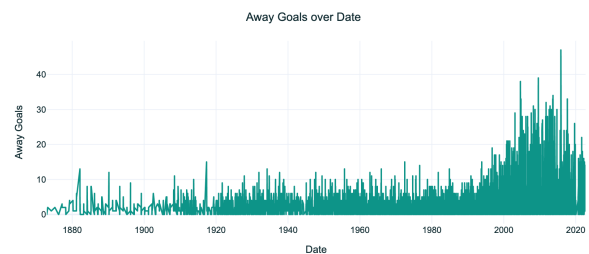


Figure 3.7 – Buts à l'extérieur au fil du temps.

3.7 Analyses avancées : anomalies et segments

L'agent KPI enrichit l'analyse de deux familles d'insights, implémentés dans `src/core/insights.py`.

3.7.1 Détection d'anomalies

La détection repose sur un **score z modifié** fondé sur l'écart absolu médian (MAD), robuste aux valeurs extrêmes que l'on cherche précisément à isoler (extrait 3.3).

```
1 def _modified_z(values):
2     median = np.median(values)
3     mad = np.median(np.abs(values - median))
4     if mad == 0:
5         return None # pas de dispersion : repli sur l'ecart
6         interquartile
7     return 0.6745 * (values - median) / mad
```

Extrait de code 3.3 – Score z modifié robuste (`insights.py`).

Toute valeur dont le score dépasse un seuil (3,5 par défaut) est signalée. La figure 3.11 montre la restitution dans le rapport.

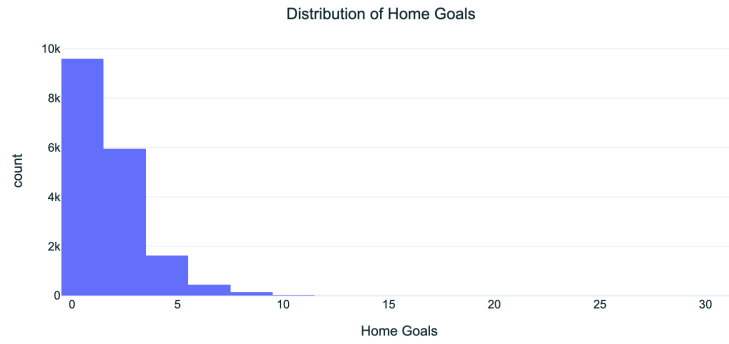


Figure 3.8 – Distribution des buts à domicile (histogramme).

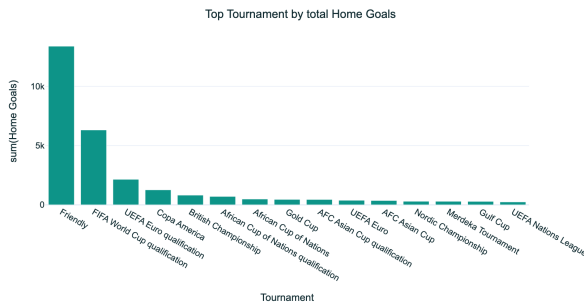


Figure 3.9 – Top tournois par buts à domicile.

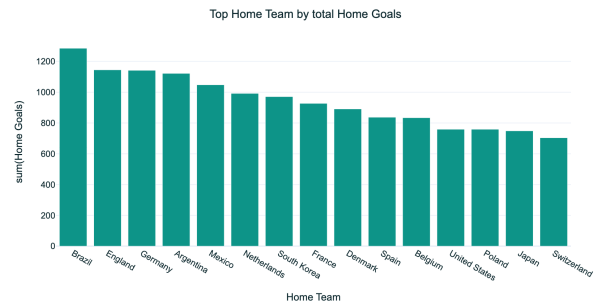


Figure 3.10 – Top équipes par buts à domicile.

3.7.2 Comparaison de segments

Pour chaque dimension de faible cardinalité et chaque mesure, le moteur compare les moyennes par groupe et retient les écarts les plus marqués (figure 3.12). Cette analyse met par exemple en évidence l'avantage du terrain dans l'étude de cas.

3.8 Orchestration et filets de sécurité

Le pipeline (`src/pipeline.py`) exécute les sept agents dans un ordre déterministe. Comme un agent peut, en pratique, omettre un appel d'outil, des **filets de sécurité** déterministes vérifient après chaque étape que les sorties attendues existent et, le cas échéant, invoquent eux-mêmes les outils manquants sous les permissions du rôle concerné. L'extrait 3.4 illustre ce dispositif pour l'étape KPI.

```
1 elif stage == "kpi":
2     if not store.get("kpis"):
3         # ... calcul des KPIs si l'agent les a omis ...
4         _safe("kpi", "compute_kpis", {"kpi_ids": top_ids})
5         # Les insights sont independants des KPIs : on garantit chacun.
6         if store.get("anomalies") is None:
7             _safe("kpi", "detect_anomalies", {})
8         if store.get("segments") is None:
```

Anomalies & Outliers			
Values flagged by a robust modified z-score (outliers are far from the typical range — not necessarily errors, but worth a look):			
Measure	Type	Flagged	Most extreme
Home Goals	value outlier	324	most extreme: value 31.0 (Australia · American Samoa · 2001-04-11) (score 20.23)
Away Goals	value outlier	101	most extreme: value 17.0 (Maldives · Iran · 1997-06-02) (score 10.79)

Figure 3.11 — Section « Anomalies » du rapport : valeurs aberrantes et exemple le plus extrême.

Segment Comparison			
How each measure's average differs across the groups of a dimension — ranked by the size of the gap between the highest and lowest group:			
Measure × Segment	Highest group (avg)	Lowest group (avg)	Ratio
Home Goals by Home Stadium or Not	1 (1.7812)	0 (1.4797)	1.204×
Away Goals by Home Stadium or Not	0 (1.2788)	1 (1.1424)	1.119×

Figure 3.12 — Section « Comparaison de segments » du rapport.

```
9 _safe("kpi", "compare_segments", {})
```

Extrait de code 3.4 — Filet de sécurité de l'étape KPI (pipeline.py).

Ce mécanisme garantit qu'une exécution produit **toujours** des livrables complets, même en cas de défaillance d'un agent — le moteur déterministe devenant le filet de dernier recours.

3.9 Publication des livrables

L'agent de **Publication** assemble le tableau de bord interactif (`dashboard.html`) et fige l'état complet de l'exécution (`state.json`). L'agent **Reporter** produit ensuite le **rapport décisionnel** (`report.html`), structuré en sections : résumé, source des données, méthodologie, indicateurs, tableau de bord, conclusions, anomalies, comparaison de segments. Le rapport est exportable en **PDF** (impression navigateur) et en **HTML** autonome ; chaque graphique est exportable en image. La figure 3.13 montre l'en-tête du rapport avec sa barre d'actions et de navigation.

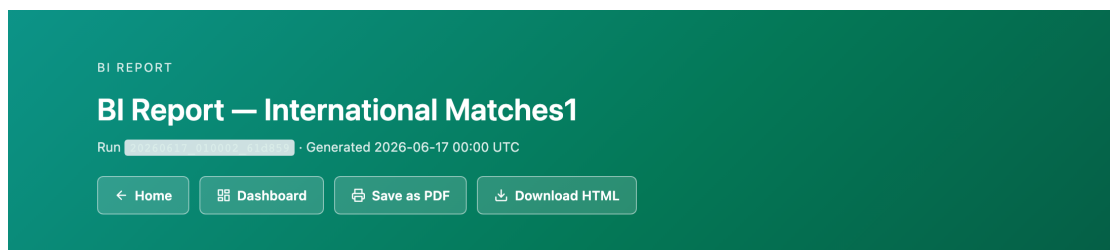


Figure 3.13 – En-tête du rapport : navigation (Accueil, Dashboard) et export (PDF, HTML).

3.10 Fiabilité : véracité des chiffres et suivi des coûts

3.10.1 Vérification des chiffres du rapport

Pour contrer les hallucinations, chaque nombre cité dans les conclusions du rapport est **vérifié** face à l'ensemble des nombres réellement calculés (valeurs des KPIs, statistiques du profil, comptages, corrélations, valeurs d'anomalies, moyennes de segments), avec une tolérance absorbant les arrondis. Le rapport affiche alors un verdict : *vérifié*, *partiel* ou *non vérifié*. La figure 3.14 montre le badge de vérification au-dessus des conclusions.



Figure 3.14 – Conclusions du rapport avec le badge de vérification des chiffres.

3.10.2 Suivi de la consommation

Le client LLM mesure, pour chaque appel, le nombre de jetons (entrée/sortie) et la latence ; le pipeline agrège ces mesures par agent et par exécution, et estime un coût à partir de prix paramétrables. Ces indicateurs sont restitués dans la vue d'exécution (figure 3.2), assurant l'observabilité du système.

Synthèse du chapitre

Le système combine un moteur déterministe généraliste (profilage, KPIs, graphiques, carte, insights) et des agents qui curent et narrent. Les filets de sécurité garantissent des livrables complets, la vérification des chiffres assure la véracité, et le suivi des coûts apporte l'observabilité. Le chapitre suivant évalue l'ensemble.

Chapitre 4

Tests, résultats et étude de cas

En bref

Ce chapitre présente la stratégie de validation du système, les résultats des tests automatisés, puis une étude de cas approfondie sur un jeu de 17 769 matchs internationaux de football, avant une discussion des apports et des limites.

4.1 Stratégie de tests

La qualité du système repose sur une suite de **22 tests automatisés** (exécutés avec `pytest`), couvrant à la fois le moteur déterministe et la logique d'orchestration. Deux principes ont guidé cette stratégie :

- **tester sans LLM** : le cœur déterministe est validé indépendamment de tout fournisseur de modèle, ce qui rend les tests rapides et reproductibles ;
- **simuler les défaillances d'agents** : un client LLM « fictif » permet de vérifier que les filets de sécurité produisent des livrables complets même lorsque les agents n'appellent aucun outil.

Le tableau 4.1 récapitule la couverture par module.

L'ensemble des 22 tests est au vert. Au-delà des tests unitaires, des **exécutions de bout en bout** sur des jeux de données réels valident l'intégration complète (du téléversement aux artefacts produits).

4.2 Étude de cas : matchs internationaux de football

4.2.1 Présentation du jeu de données

Pour valider l'approche généraliste, nous avons soumis au système un jeu de données de **17 769 matchs internationaux de football** disputés entre **1872 et 2024**. Aucune configuration spécifique n'a été fournie : le système a découvert seul la structure des données. Le profilage automatique a identifié :

- **2 mesures** : buts à domicile (*Home Goals*) et à l'extérieur (*Away Goals*) ;

Table 4.1 – Couverture des tests automatisés.

Domaine testé	Nb. de tests
Profilage des rôles de colonnes	1
Suggestion et calcul des KPIs	1
Suggestion et construction des graphiques	1
Assemblage du tableau de bord HTML	1
Permissions du serveur MCP (refus hors périmètre)	1
Vérification des chiffres (<code>validation.py</code>)	7
Relance avec incitation (<i>retry-with-nudge</i>)	2
Détection géographique & carte choroplèthe	3
Détection d'anomalies	2
Comparaison de segments	2
Extraction numérique robuste	1
Total	22

- 2 colonnes **géographiques** : équipe à domicile et à l'extérieur (reconnues comme des pays) ;
- 1 colonne **date**, et plusieurs **dimensions** (tournoi, ville, stade neutre ou non) ;
- 90 tournois, 214 équipes à domicile et 218 à l'extérieur.

La figure 4.1 montre la section « source des données » du rapport généré, incluant le schéma détecté et le résultat de la validation.

4.2.2 Résultats : indicateurs et tableau de bord

Le système a produit automatiquement un tableau de bord complet et un rapport. Parmi les indicateurs calculés :

- un total de **30 605 buts à domicile** sur l'ensemble des matchs ;
- une moyenne de **1,72** but à domicile contre **1,17** à l'extérieur ;
- un maximum de **31 buts** marqués en un seul match.

Le tableau 4.2 récapitule les principaux indicateurs produits automatiquement par le système pour ce jeu de données.

La **carte choroplèthe** (figure 3.5, chapitre précédent) révèle d'un coup d'œil la domination du **Brésil** en buts cumulés à domicile, suivi des grandes nations européennes et sud-américaines. Les **séries temporelles** (figures 3.6 et 3.7) montrent une nette intensification du nombre de buts après l'an 2000, reflet de la multiplication des rencontres internationales.

Data Source

File	international_matches1.csv
Rows	17,769
Columns	8
Measures detected	2
Dimensions detected	4
Datetime columns	1
Validation	passed
Cleaning operations applied	1

COLUMN SCHEMA

Column	Role	Type	Cardinality	Missing
ID	identifier	int64	17,769	0.0%
Tournament	high_card_dim	str	90	0.0%
Date	datetime	str	9,004	0.0%
Home Team	geo	str	214	0.0%
Away Team	geo	str	218	0.0%
Home Goals	measure	int64	21	0.0%
Away Goals	measure	int64	16	0.0%
Home Stadium or Not	boolean	int64	2	0.0%

Figure 4.1 – Section « source des données » : schéma détecté automatiquement et validation.

4.2.3 Résultats : anomalies détectées

La détection d'anomalies a automatiquement isolé des matchs au score exceptionnel. Le plus extrême correspond au match historique **Australie 31–0 Samoa américaines** (2001), record du monde de buts inscrits en un match international, correctement identifié comme valeur aberrante (score z très élevé). Côté extérieur, le match **Maldives 0–17 Iran** (1997) ressort également. Ce résultat illustre la capacité du système à faire remonter, sans aucune connaissance métier préalable, des événements réellement remarquables.

4.2.4 Résultats : avantage du terrain

La comparaison de segments a mis en évidence un phénomène bien connu des analystes du football : l'**avantage du terrain**. En segmentant selon le caractère « à domicile ou non » du stade, le système a mesuré une moyenne de buts supérieure pour l'équipe jouant chez elle (1,78 contre 1,48), confirmant quantitativement cet effet — là encore, de manière entièrement automatique.

Table 4.2 – Indicateurs clés calculés pour l'étude de cas.

Indicateur	Valeur
Nombre total de matchs	17 769
Total des buts à domicile	30 605
Moyenne de buts à domicile	1,72
Moyenne de buts à l'extérieur	1,17
Maximum de buts en un match	31
Tournois distincts	90
Équipes à domicile distinctes	214
Équipes à l'extérieur distinctes	218

4.2.5 Véracité du rapport

Enfin, le mécanisme de vérification a confirmé que **l'intégralité des chiffres** cités dans les conclusions du rapport correspondait aux indicateurs réellement calculés (verdict « vérifié », figure 3.14 du chapitre précédent), démontrant l'efficacité du garde-fou anti-hallucination.

4.3 Discussion

4.3.1 Apports

L'étude de cas confirme les promesses de l'approche :

- **généralité** : un jeu de données sportif, jamais anticipé, est analysé sans aucune configuration ;
- **pertinence** : les indicateurs, la carte et les insights produits sont immédiatement exploitables ;
- **fiabilité** : les chiffres sont exacts et vérifiés, et les livrables toujours complets grâce aux filets de sécurité ;
- **rapidité** : l'analyse complète est produite en quelques secondes, contre des heures de travail manuel.

4.3.2 Limites

Plusieurs limites subsistent :

- la détection géographique repose sur des noms de pays en anglais ; certaines entités infranationales (Écosse, Pays de Galles) ne sont pas cartographiées par la projection mondiale ;

- la qualité de la narration dépend du modèle de langage employé ;
- le système traite un fichier à la fois et ne gère pas encore les jointures entre sources ;
- l’analyse reste descriptive (pas de modélisation prédictive).

Ces limites ouvrent directement sur les perspectives présentées en conclusion.

Synthèse du chapitre

Les 22 tests valident le cœur du système, et l’étude de cas sur 17 769 matchs démontre une analyse généraliste, pertinente et fiable : carte mondiale, détection du match record 31–0, mise en évidence de l’avantage du terrain, et chiffres intégralement vérifiés.

Conclusion générale et perspectives

Ce projet de fin d’année a abouti à la conception et à la réalisation de **MCP-Powered BI Ecosystem**, une plateforme de Business Intelligence **généraliste** et **automatisée**. Partant du constat que la chaîne décisionnelle traditionnelle reste manuelle, coûteuse et spécialisée, nous avons proposé une architecture originale qui réconcilie la rigueur d’un **moteur déterministe** et le discernement d’une **équipe de sept agents d’intelligence artificielle**, le tout orchestré par le **protocole MCP**.

Les objectifs fixés en introduction ont été atteints. Une architecture multi-agents à contrôle d’accès par rôle a été mise en place, et un moteur adaptatif profile désormais n’importe quel jeu de données pour en dériver automatiquement indicateurs, graphiques et insights. Le système produit, à chaque exécution, un tableau de bord interactif et un rapport décisionnel (web et PDF) reproductibles, tandis que des mécanismes de fiabilité — filets de sécurité, vérification des chiffres et suivi des coûts — renforcent la confiance dans les résultats. L’étude de cas menée sur 17 769 matchs internationaux a enfin validé l’approche de bout en bout. Au-delà du produit, ce travail nous a permis de consolider des compétences transversales : conception logicielle en couches, intégration de modèles de langage et d’outils via un protocole standard, ingénierie de la fiabilité face à des composants non déterministes, et communication scientifique.

Plusieurs pistes d’évolution prolongent naturellement ce travail. L’ajout d’un analyste conversationnel permettrait de poser des questions en langage naturel et d’obtenir réponses et graphiques à la volée, en réutilisant le registre d’outils MCP existant. L’ingestion pourrait être étendue à de nouvelles sources de données — bases SQL, feuilles de calcul en ligne, stockage objet — ainsi qu’aux jointures multi-sources. Des analyses prédictives, telles que la prévision de séries temporelles et l’identification de facteurs explicatifs, complèteraient utilement l’analyse descriptive actuelle. Enfin, l’industrialisation de la plateforme — conteneurisation, persistance des exécutions, authentification et déploiement multi-utilisateurs — ouvrirait la voie à une mise en production.

En définitive, **MCP-Powered BI Ecosystem** démontre qu’il est possible d’automatiser la chaîne décisionnelle complète sans sacrifier la fiabilité, en assignant à chaque composant — moteur déterministe et agents IA — le rôle où il excelle. Cette complémentarité nous semble une voie prometteuse pour l’avenir des outils décisionnels.

Bibliographie

- [1] Anthropic. Model context protocol (mcp). <https://modelcontextprotocol.io>, 2024. Protocole ouvert d'intégration d'outils pour modèles de langage.
- [2] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.
- [3] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference (SciPy)*, pages 56–61, 2010.
- [4] Meta AI. The llama 3 herd of models. <https://ai.meta.com/llama/>, 2024.
- [5] Plotly Technologies Inc. Plotly : Open source graphing library for python. <https://plotly.com/python/>, 2015.
- [6] Sebastián Ramírez. Fastapi : Modern, fast web framework for building apis with python. <https://fastapi.tiangolo.com>, 2018.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

Annexe A

Annexes

A.1 Guide d'installation et d'exécution

Le système requiert Python 3.10 ou supérieur.

```
1 # 1. Environnement virtuel
2 python -m venv .venv
3 source .venv/bin/activate          # macOS / Linux
4
5 # 2. Dependances
6 pip install -r requirements.txt
7
8 # 3. Configuration (fournisseur de LLM)
9 cp .env.example .env              # renseigner GROQ_API_KEY, ou
   LLM_PROVIDER=ollama
10
11 # 4. Lancement de l'interface web
12 uvicorn src.ui.app:app --reload
13 # -> ouvrir http://127.0.0.1:8000
```

Extrait de code A.1 – Installation et lancement.

Il est aussi possible d'exécuter le pipeline en ligne de commande sur un fichier, ou de générer le jeu de données d'exemple :

```
1 # Pipeline en ligne de commande
2 python -m src.pipeline data/sample_sales.csv
3
4 # Generation d'un jeu de donnees d'exemple
5 python scripts/generate_sample_data.py --out data/sample_sales.csv
```

Extrait de code A.2 – Exécutions alternatives.

A.2 Extrait : matrice de permissions (config.yaml)

La configuration ci-dessous illustre le contrôle d'accès par rôle appliqué par le serveur MCP.

```

1 agents:
2   kpi:
3     description: >
4       Metriques et insights. Curent et calculent les KPIs, puis
5       detectent anomalies et comparaisons de segments.
6   tools:
7     - profile_dataset
8     - suggest_kpis
9     - compute_kpis
10    - detect_anomalies
11    - compare_segments

```

Extrait de code A.3 – Définition d’un agent et de ses outils autorisés.

A.3 Extrait : boucle d’appel d’outils d’un agent

```

1 for step in range(1, self.max_steps + 1):
2     response = self.llm.chat(messages, tools=tools,
3                               temperature=self.temperature)
4     if not response.tool_calls:
5         # relance avec incitation si l'agent n'a encore appele aucun
6         outil
7         if all_tool_calls or not tools or nudges_used >=
8             self.nudge_budget:
9                 final_text = response.content; break
10            messages.append(nudge_message); nudges_used += 1; continue
11 for tc in response.tool_calls:          # execution des appels
12     d'outils
13     result = self.server.call_tool(store, self.role, tc.name,
14                                     tc.arguments)
15     messages.append(tool_result_message(tc, result))

```

Extrait de code A.4 – Cœur de la boucle agentique (agents/base.py).

A.4 Arborescence du projet

```

1 src/
2   core/          # moteur deterministe : profiler, kpi_engine,
3                   # dashboard_builder, insights, validation
4   llm/           # client LLM (Groq / Ollama)
5   mcp_server/    # serveur MCP : outils + permissions
6   agents/        # les 7 agents specialises
7   ui/           # interface FastAPI + gabarits + rendu du rapport

```

```
8   pipeline.py  # orchestrateur de bout en bout
9   tests/      # 22 tests automatisés
10  config.yaml  # matrice de permissions + seuils du moteur
```

Extrait de code A.5 – Organisation des sources.